

SIA

Semantic Interaction Architecture

Průvodce k pochopení a obhajobě

Přirozené vysvětlení problému, architektury, trust modelu, času, kontextu, renderingu, buffering, kryptografie, overloadu, hardwarových limitů a hranice mezi protokolem a deploymentem.

Minimal Profile 0.4.0

Pre-standard draft · červenec 2026

Obsah

Úvod: co je SIA	4
1. Problém, který SIA řeší	4
Duplicitní logika	4
Nekonzistentní chování	4
Slabá autorita nad tím, kdo smí mluvit	4
2. Kde SIA leží v architektuře	5
Co SIA nenahrazuje	5
3. Pět hlavních částí	5
4. Deklarace významu	5
5. Typy interakcí a profil 0.4.0	6
6. Životní cyklus interakce	6
7. Osm trust kontrol	7
8. Kryptografie a přesný význam 200 ms	7
Může být kryptografie bottleneck?	7
Podporované mechanismy	7
9. Čas není jeden budget	8
10. Kontext jako více nezávislých os	8
Aplikovatelnost versus blokování	8
11. Attention model	9
12. Renderer capabilities a Translation	9
13. Delivery a occupant response jsou dvě smyčky	9
14. Buffering a overload	10
15. Scheduling a falešná kritická priorita	10
Pre-authentication admission	10
16. Elegantní implementační forma	11
17. Hardwarové bottlenecky	11

18. Fail-closed a fail-operational	11
19. Audit a soukromí	12
20. Hranice mezi SIA a deploymentem	12
21. Co SIA neprokazuje	12
22. Aktuální stav 0.4.0	13
23. Nejpravděpodobnější námitky	13
Není to jen notification service?	13
Není to jen Android Automotive template systém?	13
Proč to nedat přímo do rendereru?	13
Není SIA nový single point of failure?	14
Neohrozí kryptografie latenci?	14
Proč JSON v real-time cestě?	14
Není attention model příliš jednoduchý?	14
Proč standardizovat význam?	14
24. Formulace vhodné k obhajobě	14
Jednověťá verze	14
Delší verze	14
Co zdůraznit při technické diskusi	14
25. Mentální model na závěr	15

Úvod: co je SIA

Semantic Interaction Architecture - SIA - je návrh společné mediační vrstvy pro interakce mezi softwarovými systémy vozidla a člověkem.

Kdo smí člověku něco sdělit, co přesně dané sdělení znamená, za jakých podmínek je ještě platné, kde smí být prezentováno a jak následně prokážeme, co se skutečně stalo?

SIA se snaží oddělit význam interakce od jejího konkrétního zobrazení. Kolizní varování má mít stejnou sémantickou identitu bez ohledu na to, zda se nakonec projeví na přístrojovém štítu, hlasem, hapticky nebo kombinací více kanálů.

Nejkratší definice

SIA je úzká, deterministická a auditovatelná mediační vrstva mezi emittery a renderery. Přijímá typované interakce, ověřuje jejich autoritu, aktuálnost a kontext, vytvoří závazný renderovací plán, koordinuje doručení a uchovává důkazy o výsledku.

1. Problém, který SIA řeší

Dnešní automobilové HMI bývá postavené kolem jednotlivých obrazovek, aplikací a funkcí. Každý systém si do značné míry sám řeší prioritu, oprávnění, kontext, volbu výstupu, fallback, potvrzení i audit.

Duplictní logika

Stejné rozhodování se opakuje v clusteru, infotainmentu, hlasovém systému, aplikacích a dalších komponentách. S rostoucím počtem emitterů a rendererů vzniká síť integrací typu $N \times M$. SIA mezi ně vkládá společnou hranici, takže se emitter i renderer integrují vůči jednomu kontraktu.

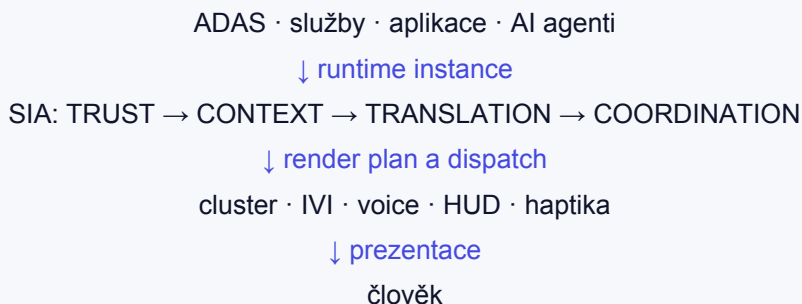
Nekonzistentní chování

Stejný význam může být na různých površích interpretován rozdílně. SIA přesouvá rozhodující pravidla do deklarovaného kontraktu. Renderer už nemá znovu interpretovat prioritu, oprávnění nebo kontextovou politiku.

Slabá autorita nad tím, kdo smí mluvit

S příchodem aplikací, cloudových služeb a AI agentů přestává stačit předpoklad, že vše uvnitř vozidla je implicitně důvěryhodné. Autentizovaná aplikace může být skutečně tou aplikací, za kterou se vydává, ale to ještě neznamená, že smí vydat kolizní varování.

2. Kde SIA leží v architektuře



Pod SIA zůstává datová a servisní infrastruktura vozidla. Nad SIA jsou konkrétní výstupy. SIA neřeší, jak ADAS zjistil překážku. Dostává už tvrzení, že nastala interakce typu `Alert.Collision.Warning`, a ověřuje, zda toto tvrzení smí vstoupit do occupant-facing prostoru.

Co SIA nenahrazuje

- operační systém, RTOS ani hypervisor;
- AUTOSAR, Kuksa, VSS nebo transportní middleware;
- HMI framework a konkrétní renderer;
- PKI, HSM a správu klíčů;
- ADAS a jeho senzorické rozhodování;
- safety case nebo regulatorní schválení celého vozidla.

3. Pět hlavních částí

Část	Úloha
Ontology Language + Schema Profile	Stabilní slovník, typy uzlů, metadata, verze a strojově čitelné kontrakty.
Trust Policy	Ověření autority, identity, čerstvosti, podpisu, revokace a replay ochrany.
Context Policy	Aplikovatelnost významu a deklarované chování při blokování.
Translation Layer	Deterministická volba způsobilých rendererů podle významu, kontextu, attention a capabilities.
Interaction Coordination Runtime	Retention, dispatch, fallback, delivery receipts, occupant response, timeouty a audit.

Translation rozhodne, co se má stát. Coordination Runtime zajistí a doloží, co se skutečně stalo.

4. Deklarace významu

Každá interakce má node declaration. Ta je autoritativním popisem významu a vzniká před runtime emisí.

- stabilní identita a typ uzlu;
- cílová role a priorita;

- povolené třídy aktérů;
- maximální ingress age a sémantická platnost;
- kontextová politika a attention metadata;
- preferované nebo povinné renderery;
- delivery timeout a success policy;
- occupant-response pravidla a privacy klasifikace.

Runtime instance nese fakta. Deklarace nese autoritu.

Emitter nesmí v jednotlivé zprávě zvyšovat prioritu, volit renderer, měnit retention ani si vynucovat acknowledgement. Runtime obálka je uzavřená. Neznámé authority-changing pole se odmítne, protože jeho tiché ignorování by skrylo pokus o eskalaci.

5. Typy interakcí a profil 0.4.0

Plná architektura počítá s Event, Action, State a Task. Event se dělí na Alert a Notification. Minimal Profile 0.4.0 skutečně emituje pouze Alert a Notification.

Typ	Význam v 0.4.0
Alert	Bezpečnostně významná systémová událost. Může mít never_block a samostatnou occupant response.
Notification	Informační sdělení s explicitním drop, defer nebo coalesce při blokování.
Action / State / Task	Architektonicky rezervované pro budoucí profily; aktuální output-delivery kontrakt se na ně nesmí mechanicky přenést.

Omezený rozsah je záměr. Profil je dost malý na implementaci a testování, ale současně procvičuje trust, kontext, retention, translation, delivery, response i audit.

6. Životní cyklus interakce

Fáze	Co se děje
1. Declare	Předem se schválí význam, autorita, čas, kontext, renderery a response kontrakt.
2. Emit	Emitter vytvoří uzavřenou runtime instanci s payloadem, identitou, časem, nonce a kryptografickými vazbami.
3. Verify	Trust Policy provede osm kontrol. Selhání zastaví interakci před Translation.
4. Translate	Context a Translation vytvoří deterministický render plan.
5. Dispatch	Runtime vytvoří časově omezený pokus a přijímá autentizované delivery evidence.
6. Close	Interakce se uzavře doručením, reakcí, timeoutem, rejection, expirací nebo retention outcome.

Každý terminální výsledek má stabilní reason code a auditní stopu. Held instance není terminální - čeká na nový kontext a před uvolněním se znovu ověřuje.

7. Osm trust kontrol

#	Kontrola	Co chrání
1	Envelope + payload schema	Priority, renderer a policy injection.
2	Declaration digest	Záměnu nebo oslabení uzlu.
3	Actor authority	Emisi významu aktérem bez oprávnění.
4	Signature / authenticator	Neověřený původ.
5	Ingress freshness	Opožděné tvrzení vydávané za aktuální.
6	Nonce replay protection	Opakované použití platné zprávy.
7	Revocation status	Použití zrušeného klíče, credentialu nebo session.
8	Semantic validity	Prezentaci významu, který již vypršel.

Obhajovací pointa

Podpis je pouze jedna z osmi kontrol. Podepsaná zpráva může být stále neautorizovaná, stará, replayovaná, revokovaná nebo navázaná na jinou deklaraci.

8. Kryptografie a přesný význam 200 ms

Pozor na terminologii

Hodnota 200 ms v referenčním kolizním varování není attention budget. Je to `max_ingress_age_ms` - maximální stáří autentizovaného tvrzení při přijetí Trust Policy.

Attention metriky jsou jiný kontrakt. Referenční kolizní uzel může mít odhad glance demand kolem 800 ms, ale to neříká, kolik času má runtime na rozhodnutí.

Může být kryptografie bottleneck?

Ano, ale nejde pouze o dobu samotného algoritmu. Podpis, fronta před HSM, transport, parsování, validace i verifikace spotřebovávají freshness od okamžiku attestation timestamp.

Správná otázka tedy není, zda je ES256 rychlé, ale jaká je tail latency celého autentizačního úseku včetně HSM contention, scheduleru, IPC, cache missů a queue residence.

Podporované mechanismy

- ES256 pro veřejný podpis v JOSE raw formátu;
- EdDSA jako další asymetrická varianta;
- HMAC-SHA-256 pro deploymentově vytvořenou a revokovatelnou session.

SIA 0.4.0 nedefinuje přenosný session-establishment protokol. Provisioning, lifecycle a revokace session zůstávají odpovědností deploymentu.

9. Čas není jeden budget

Pojem	Otázka
Ingress freshness	Jak staré je autentizované tvrzení při přijetí Trust Policy?
Semantic validity	Dokdy interakce stále popisuje užitečný aktuální význam?
Retention TTL	Jak dlouho smí runtime držet blokovanou interakci?
Delivery timeout	Dokdy musí skončit konkrétní dispatch attempt?
Occupant-response timeout	Jak dlouho je po delivery success otevřené response okno?
Attention metrics	Jakou kognitivní nebo vizuální zátěž prezentace předpokládá?

Tyto hodnoty se nesmějí prezentovat jako jeden aditivní end-to-end budget. Deployment musí samostatně prokázat, že worst-case autentizace, queueing, rozhodnutí, dispatch, renderer time-to-indication a fallback reserve vejdou do sémantické platnosti.

10. Kontext jako více nezávislých os

SIA odmítá složený štítek typu `vehicle_state = charging`, protože `charging` říká něco o energii, ale neříká, že vozidlo není v ohrožení zvenčí.

Osa	Příklad významu
<code>motion_state</code>	stojí / pohybuje se / neznámé
<code>operating_mode</code>	parked / driving / service
<code>energy_state</code>	charging / discharging / neznámé
<code>road_type</code>	city / highway / neznámé
<code>driver_state</code>	attentive / distracted / neznámé
<code>occupancy</code>	ověřená množina obsazených rolí

Každá osa nese hodnotu, čas pozorování, zdroj a confidence. Policy definuje maximální stáří, minimální confidence a zacházení s neznámou hodnotou. Nejistota nesmí systém učinit benevolentnějším: používá se `safe_worst_case` nebo `fail_closed` s deploymentovým fallbackem.

Aplikovatelnost versus blokování

Aplikovatelnost říká, zda význam v daném kontextu dává smysl. Blokování říká, zda aplikovatelná interakce může být právě teď prezentována.

Kolizní varování zůstává aplikovatelné i při nabíjení, protože do vozidla může narazit jiný automobil. Lane-departure warning při parkování aplikovatelné není a nemá se odložit na později.

11. Attention model

SIA netvrdí, že samotná hodnota glance time zaručuje bezpečnost. Attention metadata jsou auditovatelné proxy veličiny: odhadovaný glance time, mean single glance, počet kroků, cognitive load a dostupnost voice alternativy.

Přínosem je, že attention předpoklad přestává být skrytý v GUI kódu. Translation jej může porovnat se schopnostmi rendereru a deployment může zpětně doložit, z čeho rozhodnutí vycházelo.

Kalibrace vůči eye-trackingu, simulátoru, occlusion testingu a regulačním metodikám zůstává empirickou produkční prací. U never_block interakce navíc attention rozpočet nesmí způsobit tiché zmizení kritické zprávy.

12. Renderer capabilities a Translation

Renderer není safety relevant jen proto, že to o sobě prohlásí. Deklaruje své schopnosti a safety assurance, které musí být atestované důvěryhodnou registry autoritou deploymentu.

- druh povrchu a jeho stabilní identita;
- kapacita, textové a glance limity;
- podpora animace nebo eyes-free modality;
- safety assurance a odkaz na akceptované evidence.

Translation vytváří deterministický render plan. Právě jeden renderer je primary; ostatní mohou být required, concurrent nebo fallback. Nevybrané renderery dostávají explicitní reason code.

Stejné deklarace, kontext, policy a capability verze mají vést ke stejnému plánu.

Renderer plán nereinterpretuje. Zachovává si vizuální a akustický design, ale nepřebírá autoritu nad významem, prioritou a kontextovou politikou.

13. Delivery a occupant response jsou dvě smyčky

Důkaz	Co skutečně tvrdí
received	Renderer přijal požadavek.
presented	Renderer provedl occupant-facing výstup.
failed	Renderer ohlásil terminální selhání.
timed_out	Coordination Runtime uzavřel pokus v přesné deadline.
Occupant response	Oprávněný člověk provedl autentizovanou akci, nebo runtime uzavřel response timeout.

Zásadní rozlišení

presented neznámá, že řidič prezentaci viděl, pochopil nebo potvrdil. Presentation, awareness, comprehension a acknowledgement jsou rozdílná tvrzení.

Occupant-response okno se smí otevřít až po splnění deklarované delivery-success policy. Explicitní odpověď se váže na interakci, rozhodnutí, context, presented receipts, subject role a input channel.

14. Buffering a overload

Elegantní implementace nemá jeden obecný buffer a jednu FIFO frontu. Každá fronta existuje proto, že kontrakt jí dává konkrétní význam.

Fronta/ store	Účel	Overloadchování
Ingress	Čeká na trust gate.	Quota a TRUST_REJECTED_ADMISSION.
Critical lane	never_block interakce.	Rezervovaná kapacita a safety fallback.
Deferred retention	Blokované defer instance.	TTL, kvóty, deterministická evikce.
Coalescing store	Poslední hodnota pro kanonický klíč.	Atomická náhrada, starší → superseded.
Dispatch	Připravené renderer attempts.	Deadline-aware ordering.
Receipt / response	Návrat reality do runtime.	Vysoká priorita.
Audit	Persistovaná evidence.	Bounded, podle potřeby asynchronní.

Fronta nerozhoduje o významu.

Fronta sama nesmí usoudit, že interakci zahodí, sloučí nebo odloží. Povolené chování určuje deklarace: drop, defer, coalesce nebo never_block.

Overload musí skončit explicitním auditovaným výsledkem nebo safety fallbackem. Nesmí pouze prodloužit freshness či validitu tím, že paket tiše čeká.

15. Scheduling a falešná kritická priorita

Processing se dělí do oddělených traffic classes. Kritická třída má rezervované queue sloty, worker kapacitu, autentizaci, transportní kredity i fallback cestu. Nižší třída nesmí svou zátěží překročit latency band kritické třídy.

Uvnitř třídy je doporučeno earliest-deadline-first. Starší zpráva totiž nemusí mít nejkratší zbývajcí čas.

Pre-authentication admission

Před autentizací nelze věřit ani deklarovanému node_id. Útočník by jinak mohl zaplavit rezervovanou kapacitu pakety, které všechny tvrdí, že jsou kritickým varováním.

Před trust gate jsou proto dovoleny jen levné strukturální kontroly: velikost, encoding, verze, algoritmus, povinná pole, syntaktický timestamp a známý identifikátor. Claimed node identity sama o sobě nezakládá prioritu.

16. Elegantní implementační forma

```
decide(state, event, now)
→ new_state, effects[], audit_records[]
```

Nejelegantnější jádro je deterministický reducer. Uvnitř nejsou síťová volání, přímé čtení hodin, čekání na HSM, zápis na disk ani renderer I/O. Jádro dostane ověřenou událost a explicitní čas a vytvoří nový stav, efekty a auditní záznamy.

- determinismus a snadný replay;
- přirozené conformance testování;
- reprodukovatelný audit;
- oddělení protokolu od I/O a scheduleru;
- možnost použít různé OS, middleware a kryptografické backendy.

Okolní adaptéry řeší transport, HSM, časovače, persistence, watchdog, scheduling a konkrétní renderery. SIA proto nemusí definovat vlastní RTOS ani middleware.

17. Hardwarové bottlenecky

Oblast	Typický problém
HSM	Fronta, omezená concurrency, IPC a sdílení s dalšími workloady.
CPU	Scheduler jitter, cache contention, context switches, thermal throttling.
Paměť	Alokace, GC, fragmentace, lock contention.
Transport	Queue residence, kredity, partition crossing.
Renderer	Wake-up, display pipeline, audio focus, time-to-indication.
Storage	Audit flush a contention; nesmí neomezeně blokovat critical dispatch.
Čas	Secure-time dostupnost, monotonic timers, clock skew.

Nestačí měřit průměr. Podstatná je tail latency: p95, p99, p99.9, worst observed a nakonec obhajitelný worst-case bound.

Repozitářový benchmark je užitečný pro regresi a metodiku, nikoli jako ECU latency claim. Produkční důkaz vyžaduje cílový ECU/SoC, HSM, OS, transport, renderer, mixed load, thermal a degraded režimy.

18. Fail-closed a fail-operational

Princip	Použití
Fail-closed	Neautorizovaná, neověřitelná nebo neplatná zpráva nesmí projít k rendereru.
Fail-operational	Safety-relevant indikace musí mít deploymentově definovanou cestu i při výpadku mediačního runtime.

Produkční deployment musí popsat watchdog, restart, chování při nedostupném Context Policy, capability registry, secure time nebo audit storage a certifikovaný fallback či legacy path.

Současné musí zabránit nebezpečné duplicitní prezentaci, pokud se normální a fallback cesta překryjí. SIA nesmí být nezdokumentovaný single point of failure.

19. Audit a soukromí

SIA uchovává decision-relevant evidence: digests, reason codes, render plan, dispatch attempts, receipts, occupant-response outcome a vazbu na předchozí auditní záznam.

Hash-linked chain umožňuje odhalit změnu historie. Pro non-repudiation může deployment přidat podepsané checkpointy. Audit ale není automaticky úplný recovery journal; obnovení runtime může vyžadovat samostatné autentizované snapshoty nebo event journal.

Privacy princip je minimalizace. Citlivý payload lze nahradit digestem nebo redigovanou projekcí. Persistent retention vyžaduje encryption, integrity, access control a explicitní privacy policy.

20. Hranice mezi SIA a deploymentem

SIA definuje	Deployment definuje
Sémantické typy a kontrakty	Konkrétní trust anchors a key lifecycle
Autoritativní deklarace	HSM a autentizační placement
Trust a digest bindings	Session establishment
Časové a kontextové pojmy	Secure-time infrastrukturu a clock policy
Retention disposition	Kapacity, kvóty a latency bandy
Render plan a delivery evidence	Transport, encoding a renderer integraci
Occupant response a reason codes	Safety fallback, watchdog a restart
Conformance požadavky	Attention kalibraci, privacy a safety case

Tato hranice není únik od odpovědnosti. SIA požaduje, aby deployment své mechanismy doložil, ale nepředepisuje všem výrobcům stejný HSM, RTOS nebo fallback architekturu.

21. Co SIA neproказuje

- pravdivost senzorových dat a správnost ADAS rozhodnutí;
- nekompromitovaný renderer nebo operační systém;
- regulatory compliance či functional safety celého vozidla;
- že člověk prezentaci zaznamenal a pochopil;
- produkční latenci nebo platnost attention modelu;
- dostupnost celé platformy.

Interaction integrity

SIA prokazuje užší vlastnost: interakce vstoupila do occupant-facing prostoru s deklarovaným významem, od oprávněného aktéra, v platném čase a kontextu, byla přeložena podle závazného kontraktu a její lifecycle outcome je doložitelný.

22. Aktuální stav 0.4.0

SIA 0.4.0 je pre-standard draft, nikoli schválený standard ani production-ready runtime.

Co existuje	Co ještě vyžaduje produkční práci
Core Specification a strict JSON Schemas	Produkční runtime a platformní integrace
Podepsané příklady a crypto vectors	Cílové trust anchors a key operations
Conformance vectors a reason codes	HW benchmarky a worst-case evidence
Threat model a source-of-truth mapa	Deployment safety a privacy case
Interaktivní dokumentace a teaching engine	Empirická attention kalibrace
Development benchmark a reference guidance	Certifikovaný fallback a recovery

Máme konkrétní, strojově čitelný a testovatelný pre-standard kontrakt, výukovou executable implementaci jeho rozhodovacího modelu a materiál pro conformance testování. Produkční runtime, HW evidence a deployment safety case jsou další krok.

23. Nejpravděpodobnější námitky

Není to jen notification service?

Ne. Notification service typicky řeší distribuci zpráv v jedné platformě. SIA řeší přenosnou autoritu významu, kontext, atestované renderery a oddělenou lifecycle evidence napříč vozidlem.

Není to jen Android Automotive template systém?

Template systém omezuje vykreslování v konkrétním ekosystému. SIA řeší, zda aktér vůbec smí emitovat daný význam a jak se tento význam koordinuje mezi různými platformami.

Proč to nedat přímo do rendereru?

Každý renderer by znovu implementoval autoritu, kontext, prioritu, retention a lifecycle. Renderer si zachovává design, ale nepřebírá sémantickou autoritu.

Není SIA nový single point of failure?

Může se jím stát špatná implementace. Proto kontrakt požaduje bounded processing, rezervovanou kapacitu, watchdog a deploymentový fail-operational fallback.

Neohrozí kryptografie latenci?

Může, pokud sdílí přetížený HSM nebo špatně umístěnou frontu. SIA zahrnuje celé čekání do freshness a vyžaduje tail-latency měření na cílovém HW.

Proč JSON v real-time cestě?

JSON/JCS je aktuální přenosný authoring a conformance profil. Produkční wire encoding lze optimalizovat, pokud zachová stejné kontrakty a deterministickou identitu.

Není attention model příliš jednoduchý?

Je deklarativní a auditovatelný, nikoli vydávaný za hotový human-factors model. Jeho kalibrace musí být empiricky ověřena.

Proč standardizovat význam?

Bez sdílené identity nelze přenositelně vyjádřit oprávnění, kontext, fallback ani evidence. Standardizace významu neznamena standardizaci vzhledu.

24. Formulace vhodné k obhajobě

Jednověťá verze

SIA je firewall a koordinátor pro význam occupant-facing interakcí: rozhoduje, kdo smí člověku co říct, za jakých podmínek, prostřednictvím kterého způsobilého výstupu a s jakým důkazem o výsledku.

Delší verze

SIA nenavrhuje další infotainment platformu. Navrhuje společný, strojově čitelný kontrakt pro occupant-facing interakce. Tento kontrakt odděluje význam a autoritu od konkrétního rendereru, ověřuje každý runtime výskyt proti podepsané deklaraci a aktuálním credentialům, váže rozhodnutí na autentizovaný kontext a atestované schopnosti rendererů a uchovává oddělené důkazy o doručení a reakci člověka.

Co zdůraznit při technické diskusi

- Podpis není autorita - je jednou z osmi trust kontrol.
- 200 ms je ingress freshness, nikoli attention budget.
- Fronty nerozhodují; pouze realizují deklarované disposition.

- Delivery receipt není occupant acknowledgement.
- Fail-closed trust musí být doplněn fail-operational safety cestou.
- SIA standardizuje ověřitelný kontrakt, nikoli RTOS, HSM nebo GUI toolkit.
- 0.4.0 je implementovatelný pre-standard draft, ne tvrzení o hotové produkční platformě.

25. Mentální model na závěr

Celou architekturu lze držet v hlavě jako posloupnost pěti otázek:

Otázka	Vrstva
Co tato interakce znamená?	Deklarace a katalog
Kdo ji smí v tomto okamžiku tvrdit?	Trust Policy
Je význam v aktuálním kontextu aplikovatelný?	Context Policy
Který způsobitý povrch jej má prezentovat?	Translation
Co se skutečně stalo a jak to prokážeme?	Coordination Runtime a audit

Pokud se při návrhu objeví otázka, kterou nelze umístit do jedné z těchto vrstev, je to signál, že buď patří deploymentu, nebo SIA potřebuje nový explicitní kontrakt. Právě tato disciplinovaná hranice je největší hodnotou návrhu.

Zdroj pravdy: SIA Core Specification 0.4.0, Appendix A, Threat Model, Source-of-Truth map a runtime reference architecture v repozitáři SIA.